CPT304 – Topic 1
Software Engineering II

1. Software Crisis

Software Crisis

- The challenges and problems faced in software development, particularly in the 1960s and 1970s, when demand for complex software systems outpaced the ability to design, implement, and maintain them effectively.

Historical Context

The Beginning of Computing 40s – 50s

- Introduction to the first electronic computers, such as ENIAC and UNIVAC, which were primarily used for scientific and military applications.
- The use of machine or assembly language, which was time-consuming and error-prone

Historical Context

The Beginning of Crisis 60s

- The rise of commercial computing and the need for more complex software systems, like payroll and inventory management.
- **Project Failures**: e.g. the IBM OS/360, delays and budget overruns, the growing gap between software demand and development capabilities.
- lack of standardized methodologies and processes.

Historical Context

The Beginning of Software Engineering 1968-69

- Coining the Term “Software Crisis” in 1968
- The NATO conference acknowledged the inability to produce reliable and efficient software on time and within budget as a significant issue.
- **Call for a New Discipline**: A disciplined approach to software development, laying the groundwork for software engineering.

Historical Context

Escalation and Response 70s

- As systems grew more complex, the crisis deepened, projects experiencing significant overruns and failures.
- Introduction of structured programming and the Waterfall model.
- Recognition of the growing burden of software maintenance, which consumed a significant portion of resources and budgets.

Historical Context

The Impact of the Crisis

- **Economic Consequences**: The software crisis led to substantial financial losses for companies due to failed projects and inefficient software.
- **Human Factors**: Stress and burnout among developers became common, as they struggled to meet unrealistic deadlines and requirements.
- **Technological Stagnation**: The inability to deliver reliable software hampered technological progress and innovation in various industries.

Historical Context

The Path Forward

- **Emergence of Software Engineering**: The crisis catalyzed the development of software engineering as a distinct discipline.
- **Focus on Quality and Process**: Introduction of quality assurance practices, project management techniques, and process improvement models.
- **Legacy and Lessons**: The software crisis highlighted the importance of understanding software complexity, effective communication, and the need for continuous adaptation of methodologies.

Key Issues

- Project overruns in time and budget
- Software that fails to meet user requirements
- Poor quality and unreliable software
- Maintenance challenges and escalating costs

2. The werewolf appears

No Silver Bullet: Essence and Accidents of Software Engineering

- published in 1986 and has since become a seminal work in understanding the inherent challenges of software development.
- **Author Background**: Frederick P. Brooks, a renowned computer scientist

Core Thesis

- **No Silver Bullet**: Brooks argues that there is no single breakthrough—no “silver bullet”—that will dramatically improve software development productivity or reliability by an order of magnitude within a decade.
- **Essence vs. Accidents**: He distinguishes between the essential difficulties of software engineering and the accidental difficulties.

"*…become a monster of missed schedules, blown budgets, flawed products …*", in short "*a werewolf*" the solution to which is a "*silver bullet*" that " *…makes software costs drop as rapidly as computer hardware costs …*".

First published as: Brooks,F.P. (1986) "*No Silver Bullets*", Proceedings of the IFIP Tenth World Computing Conference, (ed.) H.-J. Kugler, pp 1069-79]

Silver Bullet

- Brooks uses the "silver bullet" as a metaphor for a single technology or management technique that could magically "lay to rest" the monsters of missed schedules, blown budgets, and flawed products in software engineering.
- He defines a true silver bullet as something that, by itself, promises at least an order-of-magnitude (tenfold) improvement in productivity, reliability, or simplicity.

Essence

- This is the inherent, irreducible "hard part" of software.
- It is the abstract "construct of interlocking concepts"—the data sets, relationships, and algorithms—and the difficult task of specifying, designing, and testing that conceptual construct.
- Brooks identifies four inherent properties of this essence that make it difficult to master: complexity, conformity, changeability, and invisibility.

Accident

- Difficulties that are not intrinsic to the nature of software but are instead byproducts of the methods and environments in which software is developed.
- Examples include syntax errors, slow compilation times, and manual memory management.

Targeting Essence or Accident ?

- Moving from Assembly to Python
- Domain-Driven Design
- Cloud Infrastructure
- Test-Driven Development

Building Software is hard

- *I believe the hard part of building software to be the specification, design, and testing of this conceptual construct, not the labor of representing it and testing the fidelity of the representation. [Brooks]*

Essential Difficulties 1

- Complexity
  - Vast number of states and interactions within the system.
  - An essential property, not accidental
  - Difficulties: -
    - Communication
    - Understanding all states, unreliable
    - Extending functions without side-effect
    - Hidden states that constitute security trapdoors

Essential Difficulties 2

- Conformity
  - Software must conform to human institutions and systems, which are themselves complex and ever-changing.
- Software must conform because: -
  - It is the most recent arrival on the scene
  - Perceived as the most conformable

Essential Difficulties 3

- Changeability
  - Software is subject to continuous change due to evolving requirements, technologies, and environments.
  - All successful software get changed: -
    - It is being used at the edge of or beyond the original domain
    - Survives beyond the normal life of the machine it was written for

Essential Difficulties 4

- Invisibility
  - Unlike physical systems, software lacks a physical form, making it difficult to visualize and conceptualize.

Essential Difficulties – Case Studies

- The Knight Capital Group (2012)
  - A deployment error that led to $440 million loss in 45 minutes
  - Repurpose an old binary flag to activate an updated module. In the old code, the flag is used to activate an unwanted module (Power Peg).
  - 7 servers were updated with new code except the 8th server still using the old code unknowingly.
  - Search in YouTube for “The knight capital deployment error”

Essential Difficulties – Case Studies

- Complexity
  - The "state" of the system was unintended: a mix of 7 updated servers and 1 legacy server. The developers failed to account for the **interlocking complexity** of how a single repurposed flag would behave if the environment wasn't perfectly uniform.
- Invisibility
  - Unlike a bridge where you can see a missing support beam, Knight Capital's engineers couldn't "see" that the 8th server was running old code.

Essential Difficulties – Case Studies

- Conformity
  - Knight Capital had to change its software to conform to the NYSE's new RLP rules. This external pressure to conform often leads to "accidental" shortcuts—like repurposing an old flag rather than designing a clean, new interface.
- Changeability
  - The NYSE changed its rules for trading. If Knight Capital's system had been a hardware circuit, they might have said, "We can't change this fast enough."
  - Because it was software, there was an assumption that it *could* and *should* be changed instantly to meet the deadline. This pressure to exploit software's changeability is exactly what led to the rushed, manual deployment that skipped the 8th server.

Accidental Difficulties

- Accidental difficulties are the challenges that arise from the current state of technology, tools, and practices used in software development.
- These difficulties are not intrinsic to the nature of software but are instead byproducts of the methods and environments in which software is developed.

Exercise

- Categorize each as either an Essential Difficulty or an Accidental Difficulty, and provide a brief justification
  - Determining the interlocking business rules for a cross-border tax calculation engine.
  - Configuring the YAML files for a Kubernetes cluster to ensure high availability.
  - Resolving a "Segmentation Fault" caused by manual memory management in C++.
  - Mapping out the conflicting requirements between three different stakeholders for a new hospital management system.
  - Optimizing the syntax of a Python script to run faster on a specific legacy processor.

Exercise

- Software complexity leads to many difficulties, what are they?
- In your opinion, why is software perceived as the most conformable element?
- Define essential difficulties and accidental difficulties in the context of software engineering. Provide two examples of each type of difficulty.

3. Past Breakthroughs

A successful past (in 1986)

- High level languages
  - Most important productivity development
  - Reduces accidental complexity
- Time sharing and development interactivity
  - Immediacy allows concentration
- Unified programming environments
  - e.g. Unix, provides a workbench and tools

Lethal Silver ?

- Better HLL ?
- Object Oriented programming ?
- Artificial intelligence

Lethal Silver ?

- Expert systems
- “Automatic” programming
- Graphical programming

Lethal Silver ?

- Program verification
- Environment and tools
- Workstations

Lethal Silver - Discussion

- Brooks dismissed Artificial Intelligence and Expert Systems as "Lethal Silver" in 1986.
- Today, Large Language Models (LLMs) like GitHub Copilot are often framed as a revolution

- Does AI solve Essential or Accidental difficulties?

4. The Promising Attacks

Build vs. Buy

- Not to build, to buy
- Off-the-shelf for mass market
- Immediate delivery, with less errors
- Low cost, cost distributed among users
- Example:
  - API-driven development
  - Software framework

Requirement Refinement & Prototyping

- Hardest part of building a software system is deciding precisely what to build.
- Impossible to specify completely, precisely, and correctly the exact requirements.
- Iteratively deciding "precisely what to build"

Incremental Development

- "Growing" software bit by bit rather than trying to build a complex system all at once.
- grow, don't build.

Great Designer

- Great designs come from great designers.
- Best designers produce structures that are faster, smaller, simpler, cleaner, and produced with less effort.
- Great products come from one or a few designing minds, great designers.

Relevance Today

- **Enduring Insights**: Despite technological advancements since the article's publication, the core challenges identified by Brooks remain relevant.
- **Exercise**: Discuss how contemporary practices like Agile and AI-enhanced tools are used to tackle essential difficulties identified by Fredrick Brooks.

The Laymen Terms

- **Essential difficulties** are the hurdles that are part of the nature of software. They are "essential" because even if you had a perfect computer, these problems would still exist.
- This is the **Logic.** It is the task of deciding exactly how the tax laws should be calculated, or how 100 different microservices should talk to each other without crashing.

The Laymen Terms

- **Accidental difficulties** are the hurdles we encounter because of the way we build software today. They are not part of the problem itself; they are just part of our current technology
- This includes things like managing memory manually (in C), dealing with slow compilers, inefficient server configuration, or struggling with a confusing IDE.

Essential Reading Material

- No Silver Bullet: Essence and Accidents of Software Engineering, by Frederick P. Brooks